

第11課

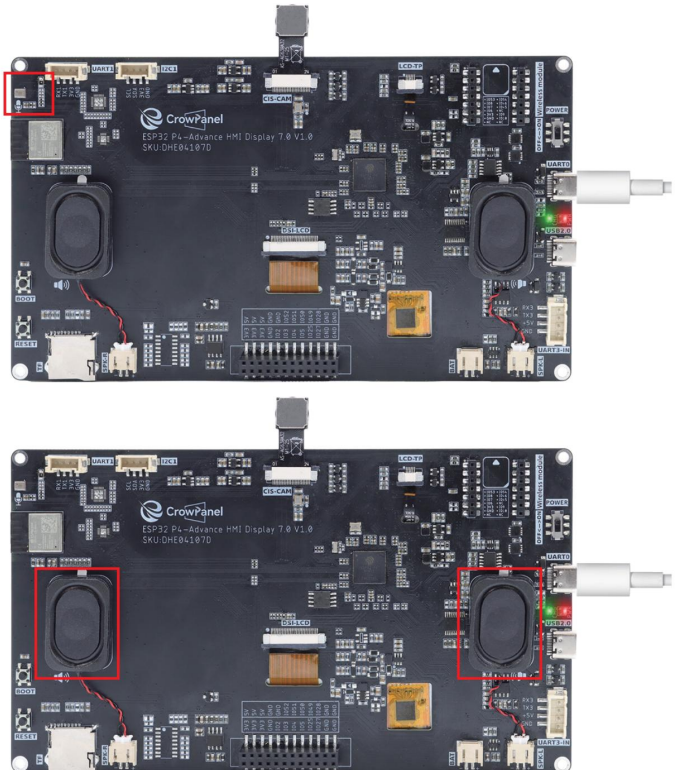
録音後の再生

導入

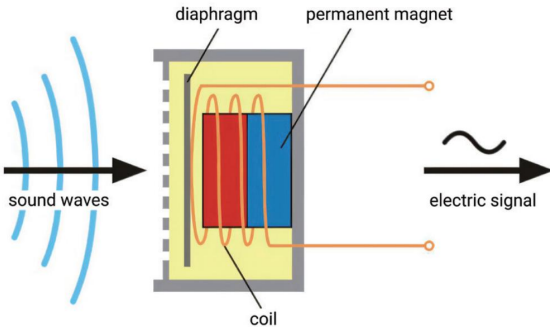
このレッスンでは、Advance-P4ボードのマイクとスピーカーの使い方を学びます。5秒間の音声を録音し、その5秒間の音声クリップを自動的に再生するプロジェクトに取り組みます。

このレッスンで使用するハードウェア

Advance-P4のマイクとスピーカー



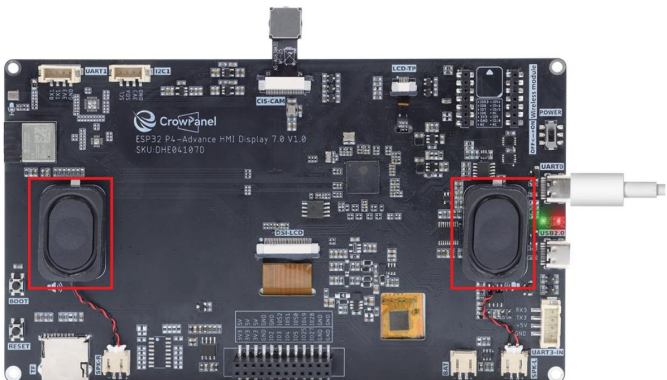
マイクとスピーカーの回路図



音声信号が音波の形で入力されると、振動板が振動します。振動板はコイルに接続されており、そのコイルは磁場中に配置された磁気コアを囲むように巻かれています。振動によってコイルが磁場で動き、磁力線を横切ります。電磁誘導の法則に従って、音声信号の変化パターンに対応する電気信号がコイル内に発生し、音声信号が電気信号に変換されます。（スピーカーの場合、これは電気信号を音声信号に変換する逆のプロセスです。通電されたコイルが磁場内で強制的に振動し、その振動によって振動板が振動して音が発生します。）

動作效果图

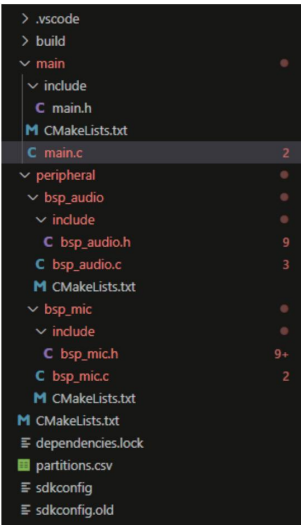
コードを実行すると、Advance-P4の近くで話せるようになります。Advance-P4は内蔵マイクを使用して5秒以内に現在の音声を録音し、自動的に再生します。



録音された5秒間の音声再生されています。

重要な説明

- このレッスンの主な焦点は、bsp_mic と bsp_audio という 2 つのコンポーネントの使用です。次に、これらのコンポーネント内の定義と関数の機能についてそれぞれ説明します。重要なのは、それらに用意されているインターフェースをいつ呼び出すかということです。
- 続いて、bsp_mic と bsp_audio。
- まず、下記のGitHubリンクをクリックして、このレッスンのコードをダウンロードしてください。
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson11-Playback_After_Recording
- 次に、このレッスンのコードをVS Codeにドラッグして、プロジェクトファイルを開きます。
- 開くと、このプロジェクトの枠組みが表示されます。



このレッスンの例では、「peripheral\」の下に「bsp_mic」と「bsp_audio」という名前の新しいフォルダが作成されます。

「bsp_audio\」フォルダ内に、新しい「include」フォルダと「CMakeLists.txt」ファイルが作成されます。（「bsp_mic」フォルダについても同様です。）

「bsp_audio」フォルダには「bsp_audio.c」ドライバファイルが含まれており、「include」フォルダには「bsp_audio.h」ヘッダーファイルが含まれています。（「bsp_mic」についても同様です。）

「CMakeLists.txt」ファイルはドライバをビルドシステムに統合し、プロジェクトが「bsp_audio.c」に記述されたオーディオ再生機能と「bsp_mic.c」に記述されたオーディオ録音機能を利用できるようにします。

「bsp_audio」のコード

まず、音声再生コンポーネントを見てみましょう。これには「bsp_audio.c」と「bsp_audio.h」という2つのファイルが含まれています。

次に、まず「bsp_audio.h」プログラムを解析します。

「bsp_audio.h」はオーディオ再生モジュールのヘッダーファイルで、主に以下の目的で使用されます。

外部プログラムで使用できるように、「bsp_audio.c」で実装されている関数、マクロ、変数を宣言します。これにより、他の .c ファイルは「#include "bsp_audio.h"」を追加するだけでこのモジュールを呼び出すことができます。

言い換えれば、モジュールの内部の詳細を隠蔽しつつ、外部に利用可能な関数や定数を公開するインターフェース層として機能する。

このコンポーネントでは、使用に必要なすべてのライブラリが「bsp_audio.h」ファイルに含まれており、一元管理が可能になっています。

```

4  /*-----Header file declaration-----*/
5  #include <string.h>
6  #include <stdint.h>
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "esp_log.h"
10 #include "esp_err.h"
11 #include "driver/gpio.h"
12 #include "driver/i2s_std.h"
13 /*-----Header file declaration end-----*/

```

- 次に、使用する必要のある変数と、その関数を宣言します。
具体的な実装は「bsp_audio.c」にあります。
- これらの宣言を「bsp_audio.h」にまとめているのは、呼び出しや管理の利便性を高めるためです。（これらの宣言の役割については、「bsp_audio.c」での使用例で説明します。）

```

15 /*-----Variable declaration-----*/
16 #define AUDIO_TAG "AUDIO"
17 #define AUDIO_INFO(fmt, ...) ESP_LOGI(AUDIO_TAG, fmt, ##_VA_ARGS_)
18 #define AUDIO_DEBUG(fmt, ...) ESP_LOGD(AUDIO_TAG, fmt, ##_VA_ARGS_)
19 #define AUDIO_ERROR(fmt, ...) ESP_LOGE(AUDIO_TAG, fmt, ##_VA_ARGS_)
20
21 #define AUDIO_GPIO_LRCLK 21
22 #define AUDIO_GPIO_BCLK 22
23 #define AUDIO_GPIO_SDATA 23
24 #define AUDIO_GPIO_CTRL 30
25
26 esp_err_t audio_init();
27 esp_err_t audio_ctrl_init();
28 esp_err_t set_audio_ctrl(bool state);
29 i2s_chan_handle_t get_audio_handle();
30
31 /*-----Variable declaration end-----*/

```

- 「bsp_audio.c」内の各関数の具体的な機能を見ていきましょう。
- 「bsp_audio.h」: このプロジェクトのカスタムオーディオモジュールヘッダーファイルは、マクロ、GPIO を定義します。
ピン、および関数宣言。

```

1  /*-----Header file declaration-----*/
2  #include "bsp_audio.h"
3  /*-----Header file declaration end-----*/
4

```

- グローバル変数 `tx_chan` を `i2s_chan_handle_t` 型で定義します。これは I2S です。
チャンネルハンドル。
- このハンドルは音声出力チャンネル (TX)を表し、以降のすべての音声再生操作はこのチャンネルを通して実行されます。

```

5  /*-----Variable declaration-----*/
6  i2s_chan_handle_t tx_chan;
7  /*-----Variable declaration end-----*/

```

- `audio_init`: この関数は、I2Sオーディオ出力チャンネルを初期化して有効にするために使用されます。サンプルレート、ビット幅、クロック、ピン設定などのパラメータを設定し、デバイスがI2Sインターフェースを介してオーディオデータを正常に再生できるようにします。

```

11 esp_err_t audio_init()
12 {
13     esp_err_t err = ESP_OK;
14
15     i2s_chan_config_t chan_cfg = {
16         .id = I2S_NUM_1,
17         .role = I2S_ROLE_MASTER,
18         .dma_desc_num = 6,
19         .dma_frame_num = 256,
20         .auto_clear = true,
21         .intr_priority = 0,
22     };
23     err = i2s_new_channel(&chan_cfg, &tx_chan, NULL);
24     if (err != ESP_OK)
25         return err;
26     i2s_std_config_t std_cfg = {
27         .clk_cfg = {
28             .sample_rate_hz = 16000,
29             .clk_src = I2S_CLK_SRC_DEFAULT,
30             .mclk_multiple = I2S_MCLK_MULTIPLE_256,
31         },
32         .slot_cfg = {
33             .data_bit_width = I2S_DATA_BIT_WIDTH_16BIT,
34             .slot_bit_width = I2S_SLOT_BIT_WIDTH_AUTO,
35             .slot_mode = I2S_SLOT_MODE_STEREO,
36             .slot_mask = I2S_STD_SLOT_BOTH,
37             .ws_width = I2S_DATA_BIT_WIDTH_16BIT,
38             .ws_pol = false,
39             .bit_shift = true,
40             .left_align = true,
41             .big_endian = false,
42             .bit_order_lsb = false,
43         },
44         .gpio_cfg = {
45             .mclk = I2S_GPIO_UNUSED,
46             .bclk = AUDIO_GPIO_BCLK,
47             .ws = AUDIO_GPIO_LRCLK,
48             .dout = AUDIO_GPIO_SDATA,
49             .din = I2S_GPIO_UNUSED,

```

- `audio_ctrl_init`: この関数は、オーディオパワーアンプの制御ピンを初期化し、出力モードとして構成して、パワーアンプのオン/オフ状態を後から制御できるようにするために使用されます。

```

66  esp_err_t audio_ctrl_init()
67  {
68      esp_err_t err = ESP_OK;
69      const gpio_config_t audio_gpio_cofig = {
70          .pin_bit_mask = 1ULL << AUDIO_GPIO_CTRL,
71          .mode = GPIO_MODE_OUTPUT,
72          .pull_up_en = false,
73          .pull_down_en = false,
74          .intr_type = GPIO_INTR_DISABLE,
75      };
76      err = gpio_config(&audio_gpio_cofig);
77      if (err != ESP_OK)
78          return err;
79      return err;
80  }

```

- `set_Audio_ctrl`: この関数は、オーディオ電源のオン/オフ状態を制御するために使用されます。
アンプ。パワーアンプ制御ピンのレベルを設定することで、パワーアンプを有効または無効にします（アクティブロー）。

```

82  esp_err_t set_Audio_ctrl(bool state)
83  {
84      esp_err_t err = ESP_OK;
85      bool status = !state;
86      err = gpio_set_level(AUDIO_GPIO_CTRL, status);
87      return err;
88  }

```

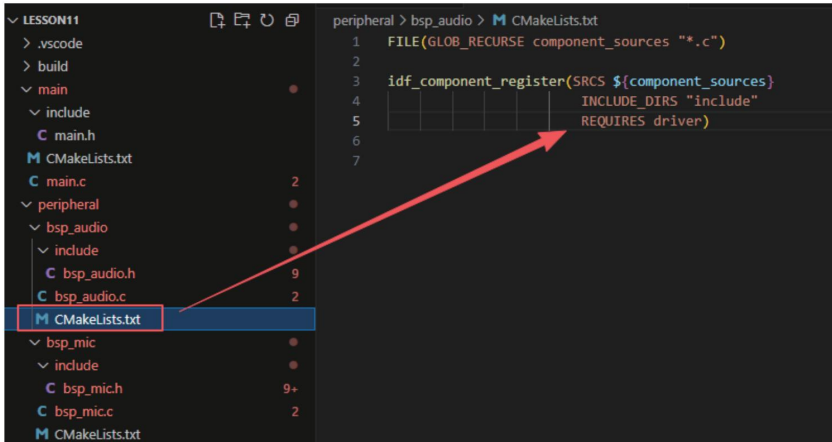
- `get_audio_handle`: この関数は、現在の I2S オーディオ出力チャネルのハンドルを取得して返すために使用され、他のモジュールがこのハンドルを使用してオーディオデータの送信または再生操作を行うことができます。

```

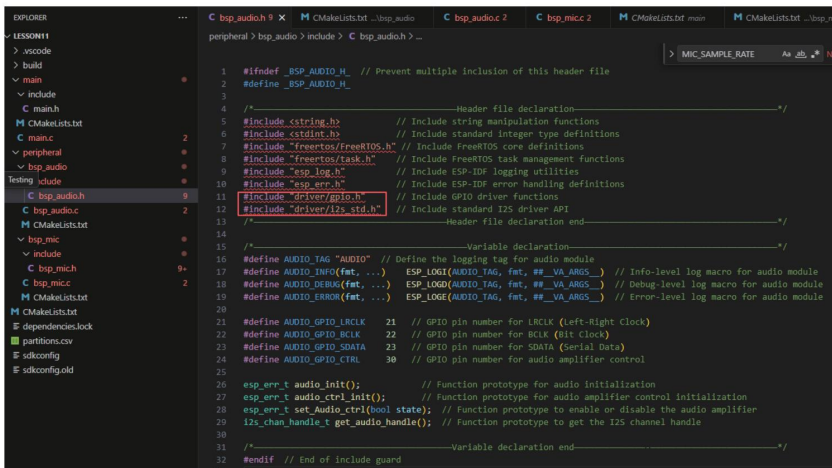
90  i2s_chan_handle_t get_audio_handle()
91  {
92      return tx_chan;
93  }

```

- これで「bsp_audio」コンポーネントの紹介は終わりです。重要なのはこれらのインターフェースの呼び出し方を知っていること。
- このコンポーネントを使用する必要がある場合は、「CMakeLists.txt」ファイルも設定する必要があります。
「bsp_audio」フォルダ内。
- このファイルは「bsp_audio」フォルダにあり、主にESP-IDFビルドシステム（CMake）に「bsp_audio」コンポーネントをコンパイルおよび登録する方法を指示する機能があります。



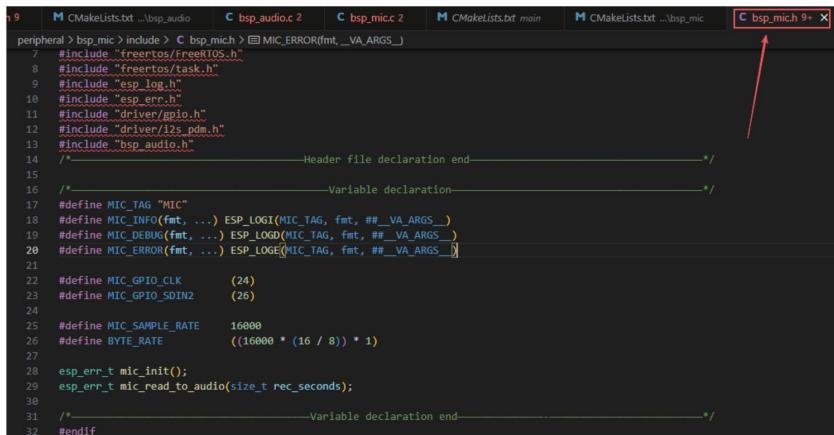
- ここで「driver」が含まれている理由は、「bsp_audio.h」でそれを呼び出しているためです。
(その他のライブラリはシステムライブラリであり、追加する必要はありません。)



「bsp_mic」のコード

それでは、音声録音がどのように実装されているかを見ていきましょう。ここでは、「bsp_mic.c」内の関数の構成を直接検証します。

まず、「bsp_mic.h」を見てみましょう。

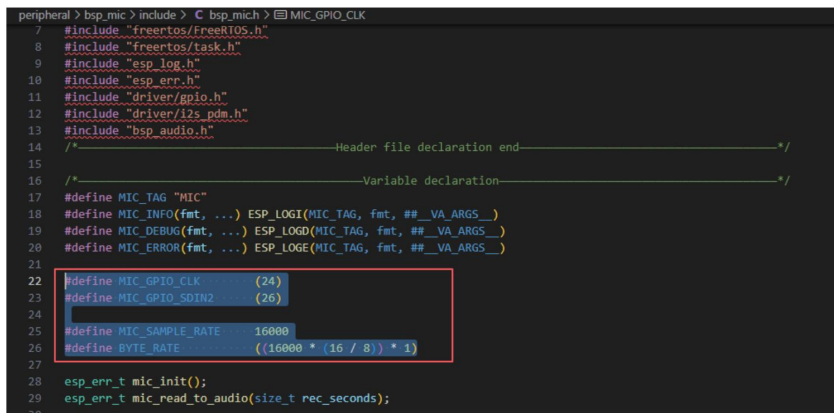


```

1 peripheral > bsp_mic > include > C bsp_mic > MIC_ERROR(fmt, __VA_ARGS__)
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "esp_log.h"
5 #include "esp_err.h"
6 #include "driver/gpio.h"
7 #include "driver/i2s_pdm.h"
8 #include "bsp_audio.h"
9
10 /*-----Header file declaration end-----*/
11
12 /*-----Variable declaration-----*/
13
14 #define MIC_TAG "MIC"
15 #define MIC_INFO(fmt, ...) ESP_LOGI(MIC_TAG, fmt, __VA_ARGS__)
16 #define MIC_DEBUG(fmt, ...) ESP_LOGD(MIC_TAG, fmt, __VA_ARGS__)
17 #define MIC_ERROR(fmt, ...) ESP_LOGE(MIC_TAG, fmt, __VA_ARGS__)
18
19 #define MIC_GPIO_CLK (24)
20 #define MIC_GPIO_SDIN2 (26)
21
22 #define MIC_SAMPLE_RATE 16000
23 #define BYTE_RATE ((16000 * (16 / 8)) * 1)
24
25 esp_err_t mic_init();
26 esp_err_t mic_read_to_audio(size_t rec_seconds);
27
28 /*-----Variable declaration end-----*/
29
30 #endif

```

- GPIO ピン: MIC_GPIO_CLK (クロック) と MIC_GPIO_SDIN2 (データ入力) は、マイクがMCUに接続される物理ピン。オーディオサンプリングパラメータ :MIC_SAMPLE_RATEはサンプリングレートを16kHzと定義し、BYTE_RATEは1秒あたりに生成されるオーディオデータの量 (32,000 バイト) を計算します。これは、後続のオーディオ処理とストレージ管理に使用されます。



```

1 peripheral > bsp_mic > include > C bsp_mic > MIC_GPIO_CLK
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "esp_log.h"
5 #include "esp_err.h"
6 #include "driver/gpio.h"
7 #include "driver/i2s_pdm.h"
8 #include "bsp_audio.h"
9
10 /*-----Header file declaration end-----*/
11
12 /*-----Variable declaration-----*/
13
14 #define MIC_TAG "MIC"
15 #define MIC_INFO(fmt, ...) ESP_LOGI(MIC_TAG, fmt, __VA_ARGS__)
16 #define MIC_DEBUG(fmt, ...) ESP_LOGD(MIC_TAG, fmt, __VA_ARGS__)
17 #define MIC_ERROR(fmt, ...) ESP_LOGE(MIC_TAG, fmt, __VA_ARGS__)
18
19 #define MIC_GPIO_CLK (24)
20 #define MIC_GPIO_SDIN2 (26)
21
22 #define MIC_SAMPLE_RATE 16000
23 #define BYTE_RATE ((16000 * (16 / 8)) * 1)
24
25 esp_err_t mic_init();
26 esp_err_t mic_read_to_audio(size_t rec_seconds);
27
28

```

- とりあえず、bsp_mic.h のマクロ定義についてはここまでとします。使用時にはこれらのマクロを変更する必要はありません。ピンは変更せず、マイクのサンプリングレートもそのままにしておいてください。次に、bsp_mic.c を見ていきましょう。
- ここでは、マイクの録音と再生を可能にする2つの機能が実装されています。I2S PDMモードを使用して、音声出力経路で出力します。

- 主に2つの機能が含まれています。マイクの初期化 (mic_init)と、録音から音声再生への変換 (mic_read_to_audio)です。

•"bsp_mic.h": マイクロフォンモジュールのヘッダーファイルで、マクロ、ピン、および関数宣言を定義します。

- rx_chan: I2S受信チャンネルハンドルを表すグローバル変数。
マイクから音声データを読み取る以降のすべての操作に使用されます。

```
peripheral > bsp_mic > C bsp_mic.c > mic_init()
1  /*-----Header file declaration-----*/
2  #include "bsp_mic.h"
3  /*-----Header file declaration end-----*/
4
5  /*-----Variable declaration-----*/
6
7  i2s_chan_handle_t rx_chan;
8  /*-----Variable declaration end-----*/
9
```

- mic_init: この関数は、マイクロフォンのI2S受信チャンネル (PDMモード)を初期化するために使用されます。サンプリングレート、DMAバッファ、GPIOピン、ハイパスフィルタ、モノラルオーディオデータフォーマットなどのパラメータを設定し、チャンネルを有効にします。これにより、システムはデジタルマイクロフォンからオーディオ信号を収集できるようになります。

```
peripheral > bsp_mic > C bsp_mic.c > mic_init()
12 esp_err_t mic_init()
13 {
14     esp_err_t err = ESP_OK;
15
16     i2s_chan_config_t rx_chan_cfg = {
17         .id = I2S_NUM_0,
18         .role = I2S_ROLE_MASTER,
19         .dma_desc_num = 6,
20         .dma_frame_num = 256,
21         .auto_clear_after_cb = true,
22         .auto_clear_before_cb = true,
23         .allow_pd = false,
24         .intr_priority = 0,
25     };
26     err = i2s_new_channel(&rx_chan_cfg, NULL, &rx_chan);
27     if (err != ESP_OK)
28         return err;
29     i2s_pdm_rx_config_t pdm_rx_cfg = {
30         .clk_cfg = {
31             .sample_rate_hz = MIC_SAMPLE_RATE,
32             .clk_src = I2S_CLK_SRC_DEFAULT,
33             .mclk_multiple = I2S_MCLK_MULTIPLE_256,
34             .dn_sample_mode = I2S_PDM_DSR_8S,
35             .bclk_div = 8,
36         },
37         /* The data bit-width of PDM mode is fixed to 16 */
38         .slot_cfg = {
39             .data_bit_width = I2S_DATA_BIT_WIDTH_16BIT,
40             .slot_bit_width = I2S_SLOT_BIT_WIDTH_AUTO,
41             .slot_mode = I2S_SLOT_MODE_MONO,
42             .slot_mask = I2S_PDM_SLOT_LEFT,
43             .hp_en = true,
44             .hp_cut_off_freq_hz = 35.5,
45             .amplify_num = 1,
46         },
47         .gpio_cfg = {
48             .clk = MIC_GPIO_CLK,
49             .din = MIC_GPIO_SDIN2,
50             .invert_flags = {
51                 .clk_inv = false,
```

mic_read_to_audio:

この機能は、マイクから指定された秒数だけ音声データを録音し、リアルタイムで再生するために使用されます。詳細なワークフローは以下のとおりです。

まず、録音時間が60秒を超えているかどうかを確認し、必要なバッファサイズを計算します。次に、I2Sインターフェースから受信した元のモノラル音声データを格納するためのread_bufと、再生用に処理されたステレオデータを格納するためのwrite_bufをSPI RAMに動的に割り当てます。

この関数は、i2s_channel_readを使用してマイクデータをブロックして読み取ります。各オーディオサンプルに対して、音量増幅（10倍）とオーバーフロー防止のためのクリッピング処理を実行します。その後、モノラルデータを左右のチャンネルにコピーしてステレオデータを作成します。

続いて、パワーアンプをオンにし（set_Audio_ctrl(true)）、処理済みの音声をI2Sオーディオ出力チャンネルを通して再生します。再生が完了すると、パワーアンプをオフにしてバッファメモリを解放し、録音と再生の全工程が安全かつ確実に行われるようにします。

(詳細な実装については、提供されているコードを参照してください。)

```

64  esp_err_t mic_read_to_audio(size_t rec_seconds)
65  {
66      esp_err_t err = ESP_OK;
67      size_t bytes_read = 0;
68      size_t bytes_write = 0;
69      if (rec_seconds > 60)
70      {
71          MIC_INFO("Exceeding the maximum recording duration");
72          return ESP_FAIL;
73      }
74      size_t rec_size = rec_seconds * BYTE_RATE;
75      i2s_chan_handle_t write_handle = get_audio_handle();
76      int16_t *read_buf = heap_caps_malloc(rec_size, MALLOC_CAP_SPIRAM);
77      if (NULL == read_buf) {
78          MIC_INFO("mic read_buf fail to apply");
79          return ESP_FAIL;
80      }
81      memset(read_buf, 0, rec_size);
82      int16_t *write_buf = heap_caps_malloc(rec_size * 2, MALLOC_CAP_SPIRAM);
83      if (NULL == write_buf) {
84          MIC_INFO("mic write_buf fail to apply");
85          return ESP_FAIL;
86      }
87      memset(write_buf, 0, rec_size * 2);
88      MIC_INFO("Start Recording %d of audio data", rec_seconds);
89      err = i2s_channel_read(rx_chan, read_buf, rec_size, &bytes_read, portMAX_DELAY);
90      if (err != ESP_OK)
91      {
92          MIC_INFO("read mic data fail");
93          return err;
94      }
95      if (bytes_read != rec_size)
96      {
97          MIC_INFO("read mic data num error");
98          return err;
99      }
100
101      int32_t data:

```

- ここでは、「bsp_audio.c」のset_audio_ctrl関数が呼び出され、パワーアップのピンがオンになり、音声再生が可能になります。

```

9  M CMakeLists.txt ...bsp_audio  C bsp_audio.c 1  C bsp_mic.c 2  M CMakeLists.txt main  M CMakeLists.txt ...bsp_mic  C bsp_mic.c 9+ X
peripheral > bsp_mic > include > C bsp_mic.h > ...
1  #ifndef BSP_MIC_H
2  #define BSP_MIC_H
3
4  /*-----Header file declaration-----*/
5  #include <string.h>
6  #include <stdint.h>
7  #include "freertos/FreeRTOS.h"
8  #include "freertos/task.h"
9  #include "esp_log.h"
10 #include "esp_err.h"
11 #include "driver/gpio.h"
12 #include "driver/i2s_pdm.h"
13 #include "bsp_audio.h"
14 /*-----Header file declaration end-----*/

```

```

9  M CMakeLists.txt ...bsp_audio  C bsp_audio.c 1 X  C bsp_mic.c 2  M
peripheral > bsp_audio > C bsp_audio.c > set_audio_ctrl(bool)
66 esp_err_t audio_ctrl_init()
67 {
68     const gpio_config_t audio_gpio_config = {
69         .pin_bit_mask = 1ULL << AUDIO_GPIO_CTRL,
70         .mode = GPIO_MODE_OUTPUT,
71         .pull_up_en = false,
72         .pull_down_en = false,
73         .intr_type = GPIO_INTR_DISABLE,
74     };
75     err = gpio_config(&audio_gpio_config);
76     if (err != ESP_OK)
77         return err;
78     return err;
79 }
80
81
82 esp_err_t set_audio_ctrl(bool state)
83 {
84     esp_err_t err = ESP_OK;
85     bool status = lstate;
86     err = gpio_set_level(AUDIO_GPIO_CTRL, status);
87     return err;
88 }

```

主な機能

- メインフォルダはプログラム実行のコアディレクトリであり、メインファイルが含まれています。
関数実行ファイル「main.c」。
- ビルドシステムの「CMakeLists.txt」ファイルにメインフォルダを追加します。

```

EXPLORER
...  C main.c  X  C bsp_illuminate.h  C bsp_illuminate.c  M CMakeLists.txt  ! idf_component.yml
LESSON4
> srcode
> build
> main
  M CMakeLists.txt
  ! idf_component.yml
  C main.c
  > managed_components
  > peripheral_bsp_illuminate
  > include
    C bsp_illuminate.h
    C bsp_illuminate.c
    M CMakeLists.txt
    M CMakeLists.txt
    dependencies.lock
    partitions.csv
    sdkconfig
    sdkconfig.old
main > C main.c > ...
1  /*-----Header file declaration-----*/
2  #include "bsp_illuminate.h" // Include LCD initialization and backlight control interface
3  #include "lvgl.h" // Include LVGL graphics library API
4  #include "freertos/FreeRTOS.h" // Include FreeRTOS core header
5  #include "freertos/task.h" // Include FreeRTOS task API
6  #include "esp_idf_regulator.h" // Include LDO (Low Dropout Regulator) API
7  #include "esp_log.h" // Include LOG printing interface
8
9  /*-----Header file declaration end-----*/
10
11 /*-----Macro definition-----*/
12 #define MAIN_TAG "MAIN" // Define log tag for this module
13 #define MAIN_INFO(fmt, ...) ESP_LOGI(MAIN_TAG, fmt, ##_VA_ARGS_) // Info level log macro
14 #define MAIN_ERROR(fmt, ...) ESP_LOGE(MAIN_TAG, fmt, ##_VA_ARGS_) // Error level log macro
15 /*-----Macro definition end-----*/
16
17 static esp_idf_channel_handle_t ldo4 = NULL; // Handle for LDO channel 4
18 static esp_idf_channel_handle_t ldo3 = NULL; // Handle for LDO channel 3
19
20 /*-----Functional function-----*/
21
22 * @brief LVGL text display function (display "Hello Elecrow")
23 */
24 static void lvel_show_hello_elecrow(void) {

```

- これはアプリケーション全体のエン트리 ファイルです。ESP-IDF には `int main()` はなく、プログラムは `void app_main(void)` から実行されます。
- まずは「main.c」について説明しましょう。
- `app_main` 関数は、アプリケーション全体のメインエン트리ポイントであり、オーディオシステムとマイクの初期化の調整、録音と再生の処理を担当します。
- まず、「main.h」への参照があります。ここでは、使用されるヘッダーファイルとマクロを格納します。「main.h」内の定義。

```

1  /*-----Header file declaration-----*/
2  #include "main.h" // Include the main header file containing declarations and macros
3
4  /*-----Header file declaration end-----*/

```

- 基本的な機能を提供するC標準ライブラリと文字列操作ライブラリを含める機能。
- タスクの作成と遅延のための FreeRTOS タスクおよびスケジューリング インターフェースを含める機能。
- ESP-IDFのログ記録およびエラー処理インターフェース (`esp_log.h`, `esp_err.h`)を含める。
- `mic_init()`, `mic_read_to_audio()`, `audio_init()`などのインターフェースにアクセスするために、マイクモジュールとオーディオモジュールのヘッダーファイルを含めます。

```

1  #ifndef MAIN_H
2  #define MAIN_H
3
4  /*-----Header file declaration-----*/
5  #include <stdio.h>
6  #include "string.h"
7  #include "freertos/freertos.h"
8  #include "freertos/task.h"
9  #include "esp_log.h"
10 #include "esp_err.h"
11
12 #include "bsp_mic.h"
13 #include "bsp_audio.h"
14 // (no extra platform headers needed for this minimal demo)
15 /*-----Header file declaration end-----*/

```

- 関数「`init_or_halt`」は、各モジュールの初期化の戻りステータスを統一的にチェックするように設計されています。これにより、重要なハードウェアまたは周辺機器の初期化が失敗した場合にシステムが実行を継続しないことが保証され、未定義の動作やハードウェアの損傷を防ぎます。
- 具体的には、モジュール名「`name`」と初期化結果「`err`」の2つのパラメータを受け取ります。「`err`」が「`ESP_OK`」と等しくない場合、初期化が失敗したことを示します。この場合、関数は「`MAIN_ERROR`」を介して詳細なエラーログ（モジュール名とエラー情報を含む）を出力し、その後、プログラムがそれ以上進まないように、各ループ反復で1秒の遅延を伴う無限ループに入ります。

```

main > C main.c > init_or_halt(const char *name, esp_err_t err)
1  /*----- Header file declaration-----*/
2  #include "main.h" // Include the main header file containing declarations and macros
3  /*----- Header file declaration end-----*/
4  /*----- Functional function-----*/
5  static void init_or_halt(const char *name, esp_err_t err) // Function to check initialization result and halt if failed
6  {
7      if (err != ESP_OK) // If initialization failed
8      {
9          MAIN_ERROR("Xs init failed: %s", name, esp_err_to_name(err)); // Log error message with component name
10         while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); } // Enter infinite loop with 1s delay to prevent program from continuing
11     }
12 }

```

•次はメイン関数「app_main」です。

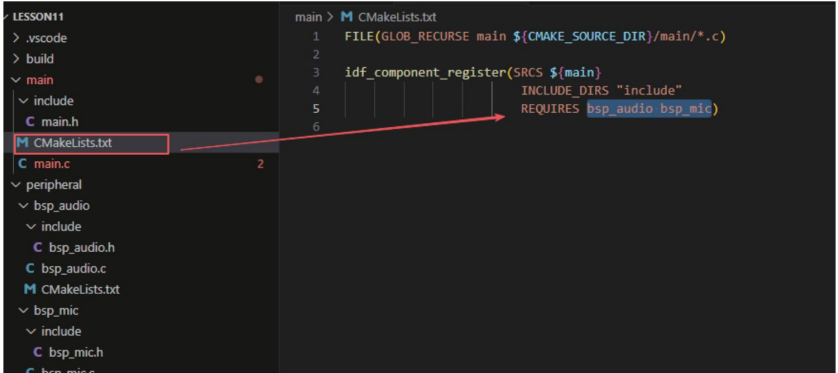
- 「app_main」関数は、アプリケーション全体の主要なエントリポイントとして機能します。オーディオシステムとマイクの初期化、および音声の録音と再生の調整を担当する。
- まず、オーディオパワーアンプとI2S再生チャンネルを初期化し、「init_or_halt」を使用して初期化が成功したかどうかを確認します。初期化に失敗すると、プログラムは無限ループに陥ります。次に、マイク入力チャンネルを初期化し、その初期化が成功したかどうかを確認します。その後、プログラムは5秒間の音声を録音し、I2S経由で再生します。この処理中に、録音および再生の状態を示すログ情報を出力し、エラーが発生した場合はエラーメッセージを記録します。
- 最後に、関数は無限ループに入り、タスクを継続させ、メインプログラムは終了せず、オーディオシステムの動作環境を維持します。全体として、この機能は音声録音と再生のための完全なサンプルワークフローを実装しています。

```

C main.c 2 x C main.h 8
main > C main.c > app_main(void)
7  static void init_or_halt(const char *name, esp_err_t err) // Function to check initialization result and halt if failed
15
16  void app_main(void) // Main entry point for the application
17  {
18      MAIN_INFO("Record 5s and playback original audio"); // Log start message for recording and playback process
19
20      // Audio amplifier and I2S playback initialization
21      esp_err_t err = audio_ctrl_init(); // Initialize audio amplifier and I2S playback control
22      init_or_halt("audio_ctrl", err); // Check initialization result; halt if failed
23      set_audio_ctrl(false); // Disable audio amplifier initially
24      err = audio_init(); // Initialize audio playback system (I2S configuration, etc.)
25      init_or_halt("audio", err); // Check initialization result; halt if failed
26
27      // Microphone initialization
28      err = mic_init(); // Initialize microphone input
29      init_or_halt("mic", err); // Check microphone initialization result; halt if failed
30
31      // Record for 5 seconds and playback
32      MAIN_INFO("Start 5s recording..."); // Log recording start message
33      err = mic_read_to_audio(5); // Record 5 seconds of audio and play it back
34      if (err != ESP_OK) // If recording or playback failed
35      {
36          MAIN_ERROR("record/playback error: %s", esp_err_to_name(err)); // Log error message
37      }
38      else
39      {
40          MAIN_INFO("Playback done"); // Log success message when playback finishes
41      }
42
43      // Keep task alive
44      while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); } // Keep the task alive indefinitely with a 1s delay
45  }

```

- 最後に、メインディレクトリにある「CMakeLists.txt」ファイルを見てみましょう。
- このCMake設定の役割は以下のとおりです。
 - main/ ディレクトリ内のすべての .c ソース ファイルを収集し、それらをソース ファイルとして使用します。コンポーネント。
 - メインコンポーネントをESP-IDFビルドシステムに登録し、カスタムコンポーネント「bsp_audio」と「bsp_mic」に依存することを宣言します。
- このようにして、ビルドプロセス中に、ESP-IDFはまずこれら2つのコンポーネントをビルドし、次にメインコンポーネントをビルドすることを認識します。



注 :以降のコースでは、新しい「CMakeLists.txt」ファイルをゼロから作成することはありません。代わりに、既存のファイルに若干の変更を加えて、他のドライバプログラムをメイン関数に統合します。

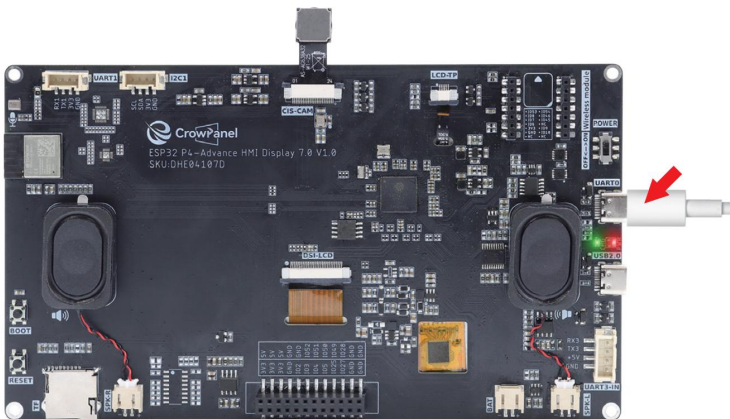
完全なコード

完全なコード実装をご覧になるには、下記のリンクをクリックしてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson11-Playback_After_Recording

プログラミング手順

- コードの準備ができたので、次に、ESP32-P4 にフラッシュして、実際の行動。
- まず、USBケーブルを使ってAdvance-P4デバイスをコンピュータに接続してください。



- フラッシュ書き込みの準備を開始する前に、コンパイル中に生成されたすべてのファイルを削除して、プロジェクトを初期の「未ビルド」状態に戻してください。(これにより、以降のコンパイルが以前の操作の影響を受けないことが保証されます。)

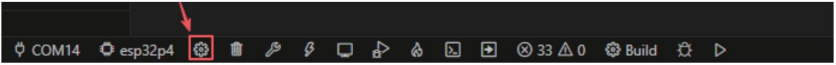
```

> vscode
> build
> main
  > include
    > C main.h
  M CMakeLists.txt
  > C main.c 2
  > peripheral
    > bsp_audio
      > include
        > C bsp_audio.h
        M CMakeLists.txt
        > bsp_mic
          > include
            > C bsp_mic.h
            M CMakeLists.txt
  M CMakeLists.txt
  > partitions.csv
  > sdkconfig
  > sdkconfigold

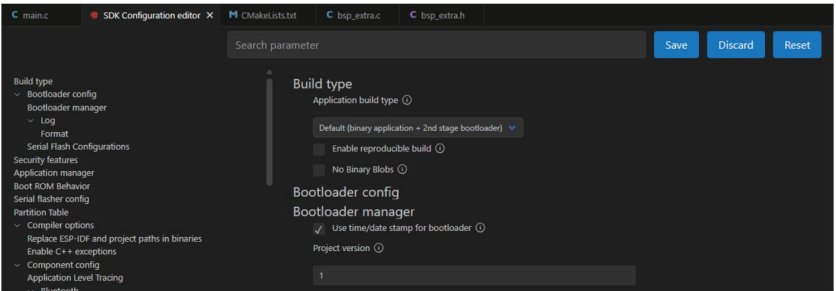
15 // Static void init() { const char *name = "main", esp_err_t err; } // Function to check initialization result and halt if failed
16 void app_main(void) // Main entry point for the application
17 {
18     MAIN_INFO("Record 5s and playback original audio"); // Log start message for recording and playback process
19
20     // Audio amplifier and I2S playback initialization
21     esp_err_t err = audio_ctrl_init(); // Initialize audio amplifier and I2S playback control
22     init_or_halt("audio ctrl", err); // Check initialization result; halt if failed
23     setAudioCtrl(false); // Disable audio amplifier initially
24     err = audio_init(); // Initialize audio playback system (I2S configuration, etc.)
25     init_or_halt("audio", err); // Check initialization result; halt if failed
26
27     // Microphone initialization
28     err = mic_init(); // Initialize microphone input
29     init_or_halt("mic", err); // Check microphone initialization result; halt if failed
30
31     // Record for 5 seconds and playback
32     MAIN_INFO("Start 5s recording..."); // Log recording start message
33     err = mic_read_to_audio(5); // Record 5 seconds of audio and play it back
34     if (err != ESP_OK) // If recording or playback failed
35         MAIN_ERROR("record/playback error: %s", esp_err_to_name(err)); // Log error message
36     else
37         MAIN_INFO("Playback done"); // Log success message when playback finishes
38
39     // Keep task alive
40     while (1) { vTaskDelay(pdMS_TO_TICKS(1000)); } // Keep the task alive indefinitely with a 1s delay
41 }
42 /* Functional function end */
43

```

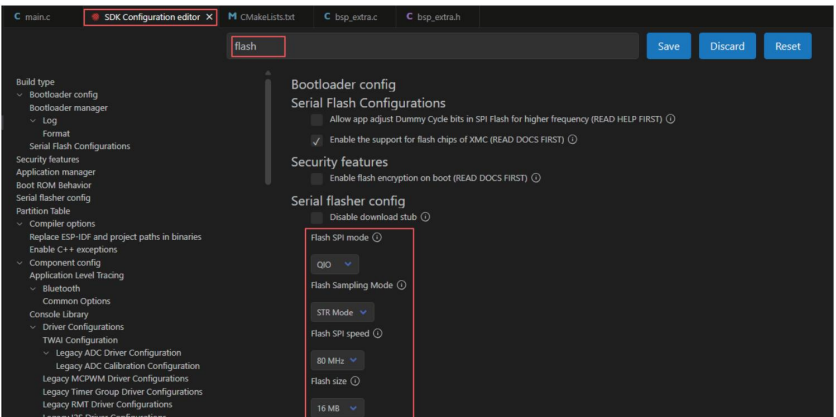
- ここでは、最初のセクションの手順に従って、ESP-IDFバージョンを選択し、コードをアップロードします。
まず、方法、シリアルポート番号、および対象チップを指定します。
- 次に、SDKを設定する必要があります。
- 下図に示すアイコンをクリックしてください。



- 短い読み込み時間の後、関連するSDKに進むことができます。
設定。



- このプロジェクトでは画面に情報を表示する必要があるため、まずレッスン7のSDK構成を参照して画面表示設定を完了し、画面が正しく機能することを確認することをお勧めします（詳細は101～103ページを参照してください）。
- 次に、検索ボックスに「flash」と入力して検索してください。（お使いのFlashの設定が私のものと同じであることを確認してください。）



•設定が完了したら、必ず設定を保存してください。

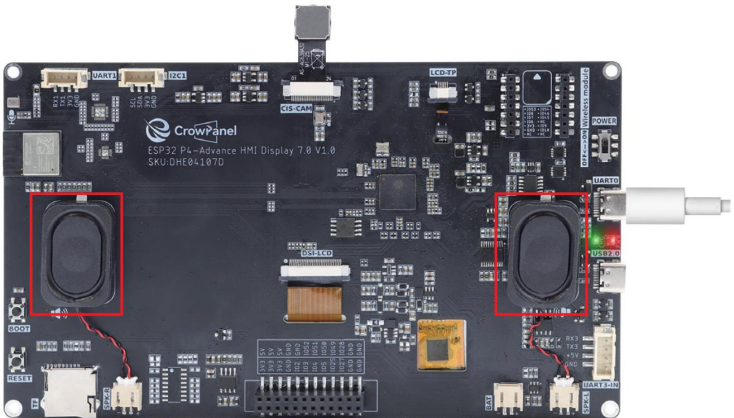
•次に、コードをコンパイルしてフラッシュします（詳細な手順は最初のレッスン）。

•ここで、非常に便利な機能をご紹介します。コンパイル、アップロード、および起動状況の監視を一度に実行できるボタンが1つあります。（ただし、コード全体にエラーがないことが確認されていることを前提としています。）



•しばらくお待ちください。コードのコンパイルとアップロードが完了し、モニターが表示されます。その後自動的に開きます。

•フラッシュが正常に完了したら、Advance-P4 デバイスの近くで話すことができます。Advance-P4は、内蔵マイクを使用して5秒以内に現在の音声録音し、自動的に再生します。



録音された5秒間の音声再生されています。